

ЛЕКЦІЯ 12

НЕЙРОННІ МЕРЕЖІ

Попередній матеріал

1. МОДЕЛІ НЕЙРОННИХ МЕРЕЖ

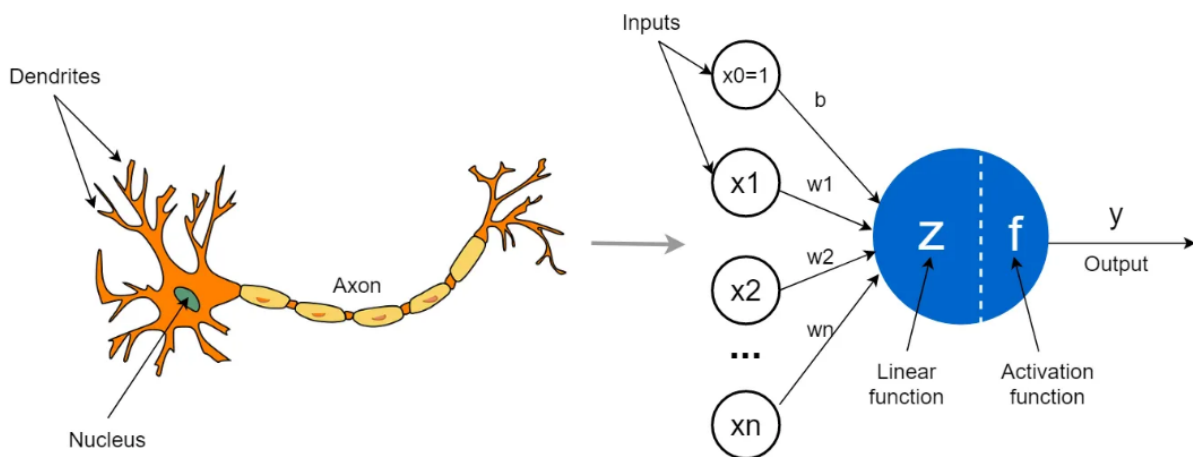
2. ПЕРЦЕПТРОН

3. БАГАТОШАРОВИЙ ПЕРЦЕПТРОН

4. КОНВОЛЮЦІЙНІ НЕЙРОННІ МЕРЕЖІ

1.МОДЕЛІ НЕЙРОННИХ МЕРЕЖ

Моделі нейронних мереж є основою сучасного машинного навчання. Вони імітують структуру та функції біологічних нейронних мереж, щоб вирішувати складні завдання. Нижче наведено основні типи моделей нейронних мереж і їхнє застосування. Нейронна мережа (або штучна нейронна мережа) — це математична модель, *натхненна роботою біологічного мозку*, яка складається з **нейронів** (вузлів) та **зв'язків** між ними (синапсів).



Штучна нейронна мережа

- Складаються з **нейронів** (вузлів), з'єднаних між собою **вагами**.
- Кожен нейрон виконує просту операцію: отримує вхідні дані, обчислює зважену суму, застосовує **функцію активації** й передає сигнал далі.
- Завдяки багатошаровим з'єднанням мережа здатна **навчатися на прикладах** і знаходити закономірності в даних.

Типи нейронних мереж

- **Перцептрон (Perceptron)** – найпростіша модель.
- **Багатошарові перцептрони (MLP, Multilayer Perceptrons)** – основа класичних ANN.
- **Рекурентні мережі (RNN)** – для обробки послідовностей (текст, мова, часові ряди).
- **Згорткові мережі (CNN)** – для аналізу зображень, відео.
- **Трансформери (Transformers)** – сучасна архітектура для роботи з мовою, зображеннями та навіть мультимодальністю (наприклад, ChatGPT побудований на трансформерах).

Що таке глибоке навчання?

Глибоке навчання (Deep Learning, DL) — підгалузь машинного навчання, яка використовує **глибокі нейронні мережі** (з багатьма шарами).

- Кожен шар поступово виділяє **рівні ознак**:
 - низького рівня (лінії, кути на зображенні),
 - середнього рівня (контури, текстури),
 - високого рівня (об'єкти, слова, сенс).

Навчання нейронних мереж

- Використовує алгоритм **зворотного поширення помилки (backpropagation)** та методи оптимізації (наприклад, **градієнтний спуск**).
- Необхідні **великі обсяги даних** і **обчислювальні ресурси** (GPU/TPU).
- Мережа поступово підлаштовує ваги, щоб мінімізувати похибку між передбаченням і реальною відповіддю.

Приклади застосування

- Розпізнавання зображень (фото, відеоспостереження, медична діагностика).
- Обробка природної мови (чат-боти, перекладачі, пошук інформації).
- Автономний транспорт (дрони, безпілотні авто).
- Біоінформатика (генетичний аналіз, моделювання білків).
- Прогнозування фінансових ринків, клімату, енергетики.

Отже, нейронні мережі — це фундаментальний інструмент ШІ, а глибоке навчання стало рушійною силою сучасної революції в AI, що дозволяє досягати надлюдського рівня у багатьох завданнях.

2. ПЕРЦЕПТРОН

Перцептрон (Perceptron) — це найпростіша модель штучного нейрона, запропонована **Френком Розенблаттом у 1958 році**.

- Має **кілька входів** ($x_1, x_2, \dots, x_n$).
- Кожен вхід має свою **вагу** ($w_1, w_2, \dots, w_n$).
- Обчислюється **зважена сума**:

$$z = \sum_{i=1}^n w_i x_i + b$$

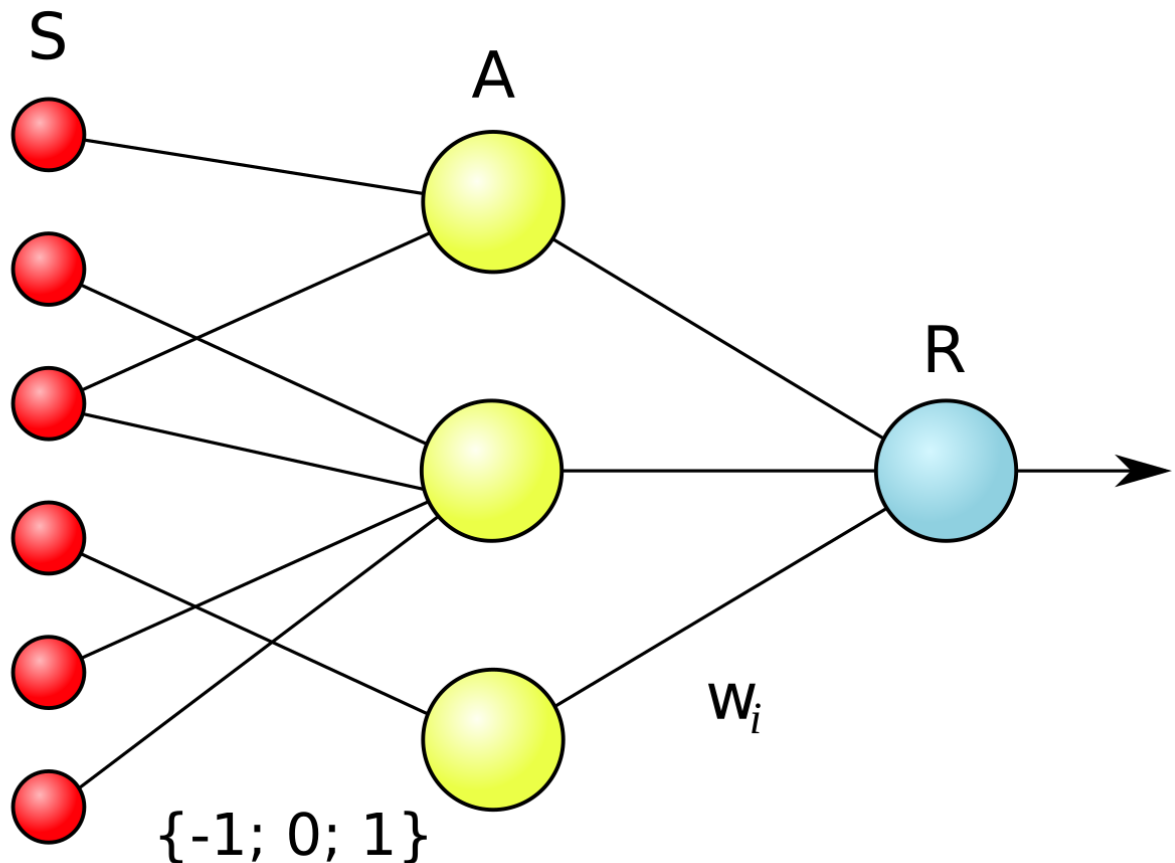
де b — порогове значення (bias).

- До результату застосовується **функція активації** (наприклад, ступінчаста), яка вирішує, чи нейрон "вмикається".

$$y = f(z)$$

- Використовується для **лінійної класифікації** (поділ об'єктів на 2 класи).

Перцептрон може розв'язувати лише **лінійно роздільні задачі**.



Логічна схема елементарного перцептрону. Ваги зв'язків S-A можуть мати значення -1 , 1 або 0 (тобто відсутність зв'язку). Ваги w_i зв'язків A-R можуть мати будь-яке значення. Якщо вага $w = -1$, це означає, що сигнал з входу **пригнічує** активацію нейрона.

*Елементарний перцептрон складається з елементів трьох типів: **S-елементів**, **A-елементів** та одного **R-елементу**. **S-елементи** — це шар сенсорів, або рецепторів. У фізичному втіленні вони відповідають, наприклад, світлочутливим клітинам сітківки ока або фоторезисторам матриці камери. Кожен рецептор може перебувати в одному з двох станів — *спокою* або *збудження*, і лише в*

останньому випадку він передає одиничний сигнал до наступний шару, асоціативним елементам.

А-елементи називаються асоціативними, тому що кожному такому елементові, як правило, відповідає цілий набір (асоціація) **S-елементів**. **А-елемент** активізується, щойно кількість сигналів від S-елементів на його вході перевищує певну величину θ .

Сигнали від збуджених А-елементів, своєю чергою, передаються до суматора R, причому сигнал від і-го асоціативного елемента передається з коефіцієнтом w_i . Цей коефіцієнт називається *вагою* А-R зв'язку.

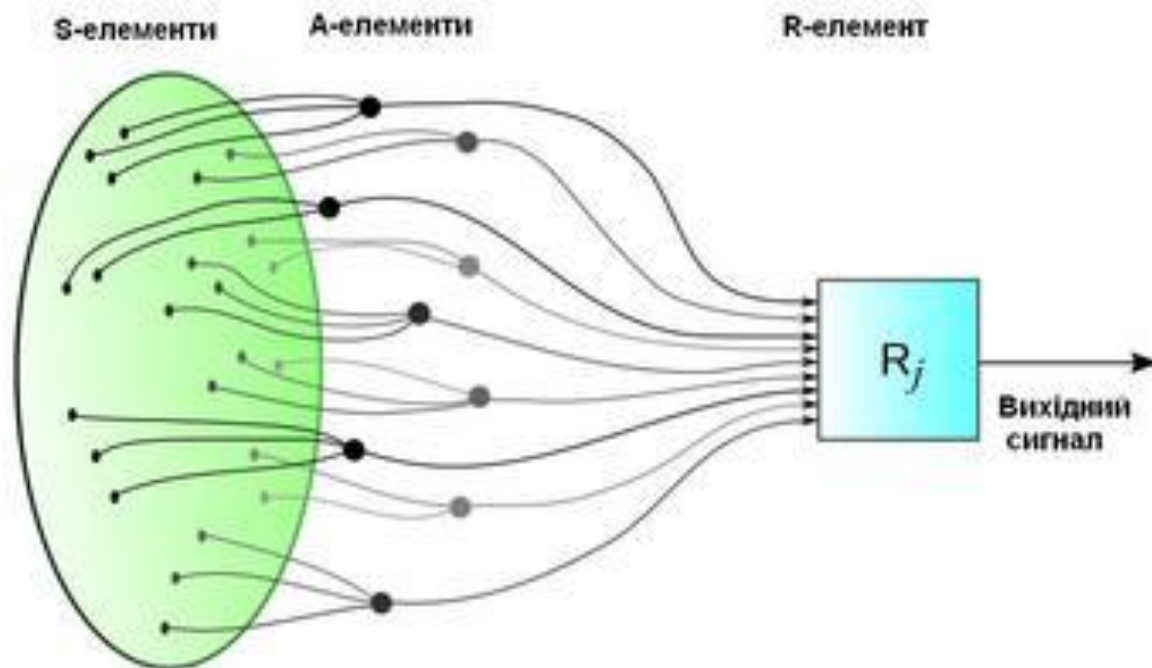
Так само як і **А-елементи**, **R-елемент** підраховує суму значень вхідних сигналів, помножених на ваги (лінійну форму). **R-елемент**, а разом з ним і елементарний перцептрон, видає «1», якщо лінійна форма перевищує поріг b , інакше на виході буде «-1». Математично, функцію, що реалізує R-елемент, можна записати так:

$$f(x) = \text{sign}\left(\sum_{i=1}^n w_i x_i - b\right)$$

Навчання елементарного перцептрона полягає у зміні вагових коефіцієнтів зв'язків А-R. Ваги зв'язків S-A (які можуть приймати значення (-1; 0; 1)) і значення порогів **А-елементів** вибираються випадковим чином на самому початку і потім не змінюються.

Після навчання перцептрон готовий працювати в режимі *розпізнавання* або *узагальнення*. У цьому режимі перцептрону пред'являються раніше невідомі йому об'єкти, й він повинен встановити, до якого класу вони належать. Робота перцептрона полягає в наступному: при пред'явленні об'єкта, збуджені **А-елементи** передають сигнал **R-елементу**, що дорівнює сумі відповідних коефіцієнтів w_i . *Якщо ця сума позитивна, то ухвалюється рішення, що даний об'єкт належить до першого класу, а якщо вона негативна — то до другого.*

Фізичний аналог перцептрона



Надходження сигналів із сенсорного поля до розв'язувальних блоків елементарного перцептрона в його фізичному втіленні.

Висновок:

Перцептрон — це "цеглинка", основа, з якої почався розвиток штучних нейронних мереж. Простий перцептрон може розв'язувати, зокрема, задачі типу AND. Що це означає на практиці?

Простий (одношаровий) перцептрон — це модель, яка має ряд входів $x_1, x_2, \dots, x_n$, зважує їх коефіцієнтами w_i , додає зсув b і застосовує **поріг (активацію)**.

$$f(x) = \begin{cases} 1, & \text{якщо } \sum_{i=1}^n w_i x_i + b \geq 0 \\ 0, & \text{інакше} \end{cases}$$

Опис: Найпростіша модель нейронної мережі, яка складається з одного шару нейронів. Вона приймає на вхід кілька сигналів, обчислює їхню вагову суму і передає через активаційну функцію.

Застосування: Використовується для задач лінійної класифікації.

[AI PR 2-1]

Програма демонструє роботу простого перцептрона

```
import numpy as np
import matplotlib.pyplot as plt

# Функція активації
def step_function(x):
    return 1 if x >= 0 else 0

# Клас перцептрона
class Perceptron:
    def __init__(self, input_size,
learning_rate=0.1):
        self.weights = np.zeros(input_size +
1) # +1 для bias
        self.learning_rate = learning_rate

    def predict(self, x):
        x = np.insert(x, 0, 1) # додаємо
bias=1
        weighted_sum = np.dot(self.weights, x)
        return step_function(weighted_sum)

    def train(self, X, y, epochs=10):
        for _ in range(epochs):
            for xi, target in zip(X, y):
                xi = np.insert(xi, 0, 1) #
додаємо bias
                prediction =
step_function(np.dot(self.weights, xi))
                error = target - prediction
                self.weights +=
self.learning_rate * error * xi
```

```

# Дані (логічна операція AND)
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 0, 0, 1])

# Створюємо і тренуємо перцептрон
p = Perceptron(input_size=2)
p.train(X, y, epochs=10)

# Вивід результатів
print("Результати після навчання:")
for xi in X:
    print(f"{xi} -> {p.predict(xi)}")

# Візуалізація
plt.figure(figsize=(6,6))

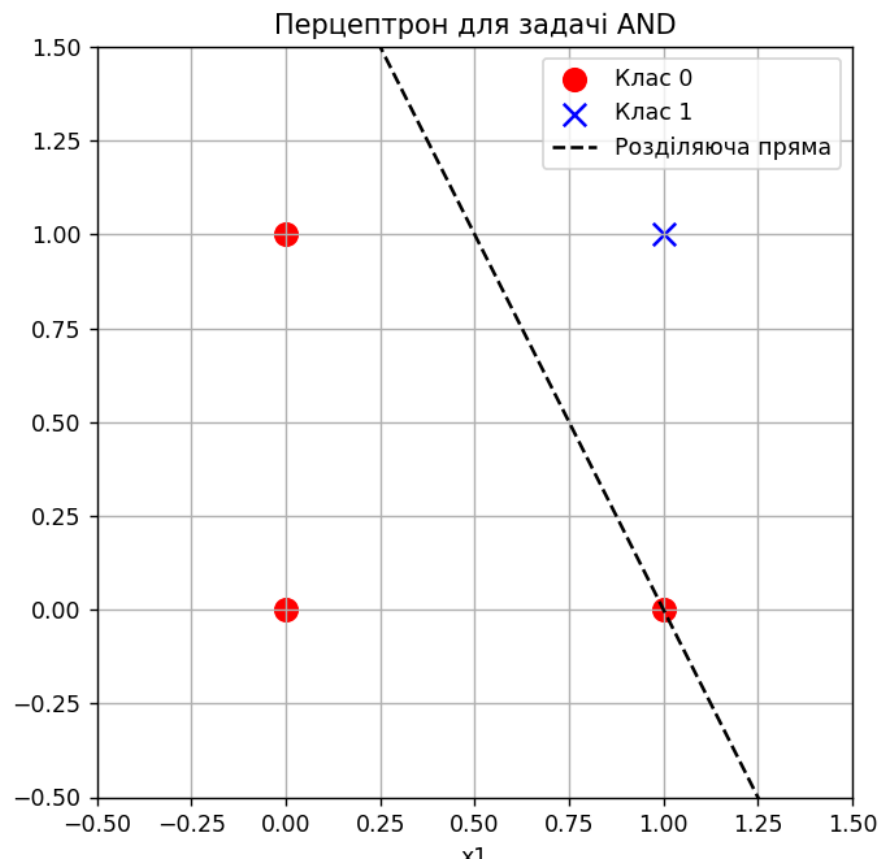
# Точки
for xi, label in zip(X, y):
    if label == 0:
        plt.scatter(xi[0], xi[1], color='red',
marker='o', s=100, label="Клас 0" if "Клас 0"
not in
plt.gca().get_legend_handles_labels()[1] else
"")
    else:
        plt.scatter(xi[0], xi[1],
color='blue', marker='x', s=100, label="Клас
1" if "Клас 1" not in
plt.gca().get_legend_handles_labels()[1] else
"")

# Побудова розділяючої прямої
x_vals = np.linspace(-0.5, 1.5, 100)
# рівняння:  $w_0 \cdot 1 + w_1 \cdot x_1 + w_2 \cdot x_2 = 0 \Rightarrow x_2 = -$ 
 $(w_0 + w_1 \cdot x_1) / w_2$ 
w = p.weights
if w[2] != 0: # щоб уникнути ділення на нуль
    y_vals = -(w[0] + w[1]*x_vals) / w[2]
    plt.plot(x_vals, y_vals, 'k--',
label="Розділяюча пряма")

```



```
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Перцептрон для задачі AND")
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.legend()
plt.grid(True)
plt.show()
```



3. БАГАТОШАРОВИЙ ПЕРЦЕПТРОН

Багатошаровий перцептрон (MLP – Multi-Layer Perceptron)

- **Опис:** Мережа, яка має кілька шарів: вхідний, *приховані (один або більше)* та вихідний. Використовує *нелінійні активаційні функції*.
- **Застосування:** Класифікація, регресія, передбачення даних.

Одношаровий перцептрон (той, що ми вже розглянули) може розв'язувати лише **лінійно роздільні задачі**. Але він не може навчитися задачам, де немає однієї прямої, яка розділить класи.

Вирішення — використати **декілька шарів нейронів**, тобто багатошарову мережу. *Багатошарові мережі (MLP) стали першим реальним інструментом для вирішення складних задач машинного навчання.*

Переваги MLP:

- Може розв'язувати задачі класифікації, регресії, розпізнавання образів, прогнозування.
- Подолала обмеження простого перцептрона.

Недоліки:

- Для глибоких мереж потрібні великі дані й обчислювальні ресурси.
- Схильність до перенавчання.

Архітектура MLP

Типова багатошарова мережа складається з:

1. Вхідний шар

- приймає дані (наприклад, використовуючи кілька входів).

2. Приховані шари

- один або більше; кожен нейрон у цьому шарі розраховує зважену суму та нелінійно активується.
- Ці шари дозволяють моделі «захоплювати» складні залежності.

3. Вихідний шар

- видає результат (наприклад, класифікація 0 або 1).

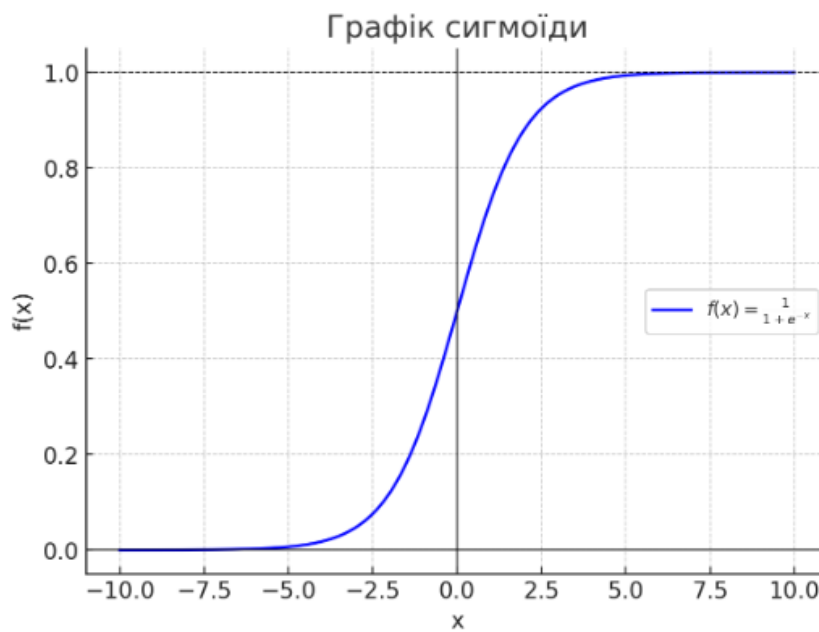
Активаци́йні функції

Без них багатошарова мережа поводитись би як один «великий лінійний перцептрон».

Популярні активації:

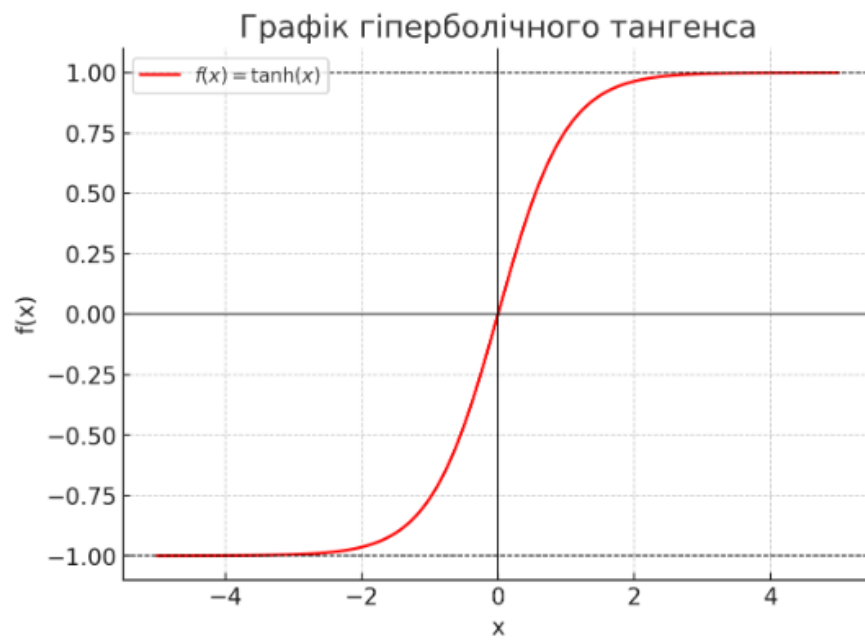
Sigmoid:

$$f(x) = \frac{1}{1 + e^{-x}}$$



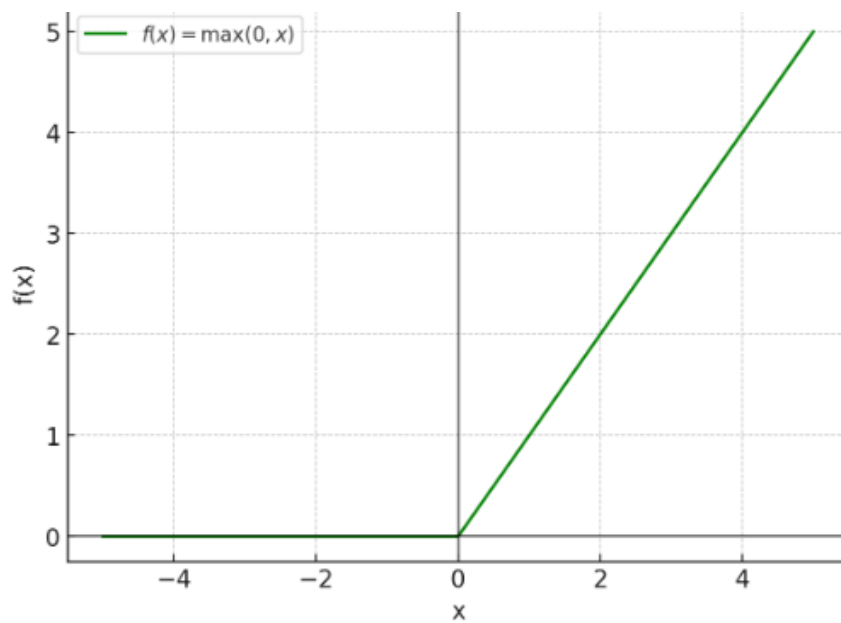
tanh:

$$f(x) = \tanh(x)$$

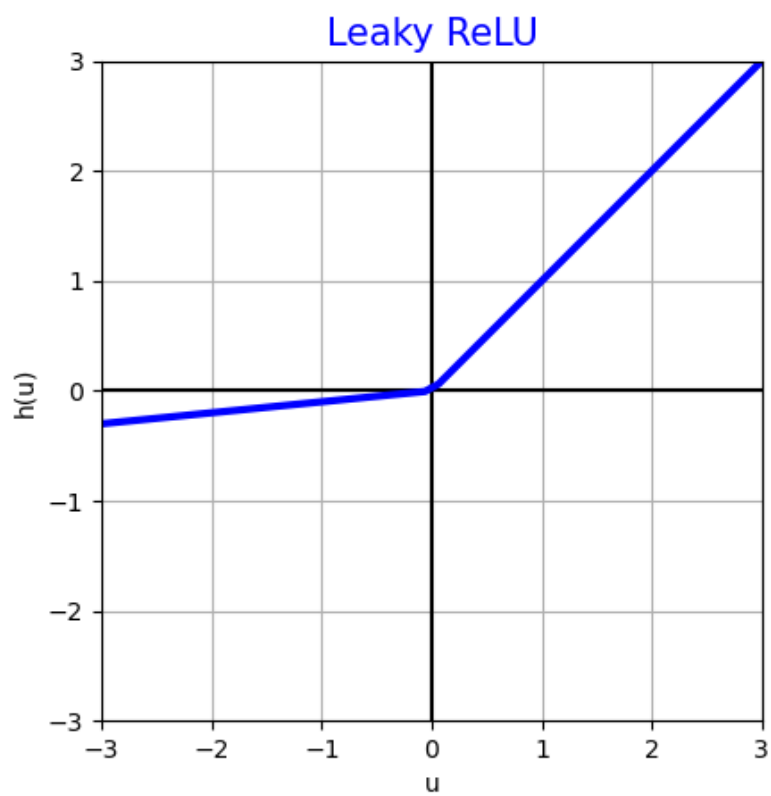


ReLU:

$$f(x) = \max(0, x)$$



LeakyReLU — модифікація ReLU для негативних значень



Як бачимо, активаційна функція — це *нелінійне перетворення*, яке застосовується після обчислення зваженої суми вхідних сигналів нейрона:

$$y = f\left(\sum_i w_i x_i + b\right)$$

Основні причини:

1. Внесення нелінійності

- Якщо в MLP не застосовувати **активаційні функції**, то вся мережа (навіть із багатьма шарами) зводиться до одного лінійного перетворення:

$$W_2(W_1x) = W_2W_1X$$

Тобто результат можна було б отримати одразу без прихованих шарів.

- Нелінійність дозволяє мережі моделювати **складні залежності**.

2. Можливість розв'язувати складні задачі

- Без активації перцептрон може реалізувати лише **лінійно відокремлювані функції** (AND, OR).
- Завдяки активаційним функціям MLP може навчитися логіці XOR, розпізнаванню образів, прогнозуванню тощо.

3. Нормування та стабілізація

- Деякі функції (наприклад, **sigmoid** або **tanh**) стискають вихід у діапазон $[0,1]$ або $[-1,1]$.
- Це допомагає уникати надто великих значень і робить навчання стабільнішим.

4. Біологічна аналогія

- У мозку нейрони також спрацьовують нелінійно: або «збуджуються», або «пригнічуються» після досягнення певного порогу.

Функція $f(x) = \max(0, x)$ бере два числа — 0 та x — і вибирає більше:

$$f(x) = \begin{cases} 0, & \text{якщо } x < 0 \\ x, & \text{якщо } x \geq 0 \end{cases}$$

Це "ламана лінія", яка йде вздовж осі x до нуля, а потім — піднімається вгору під кутом 45° .

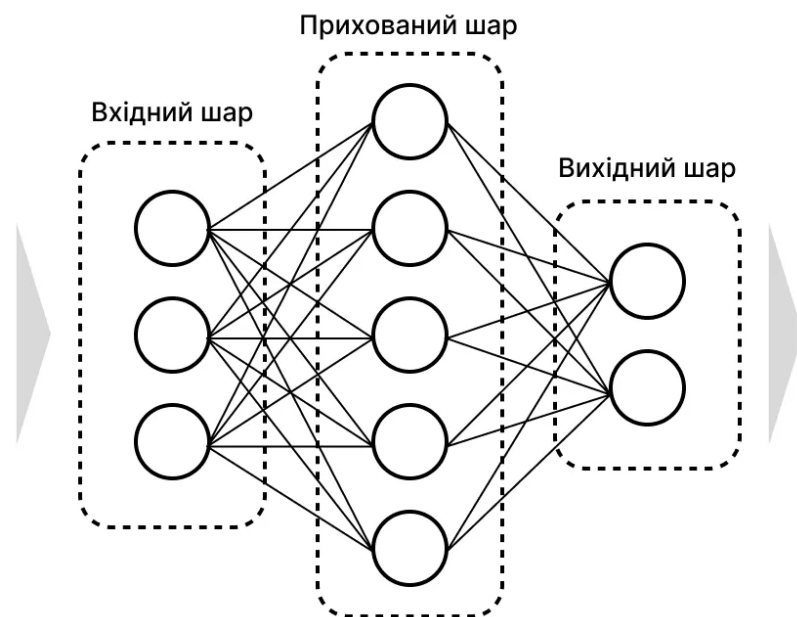
Навчання MLP

Метод називається **зворотне поширення помилки (backpropagation)**:

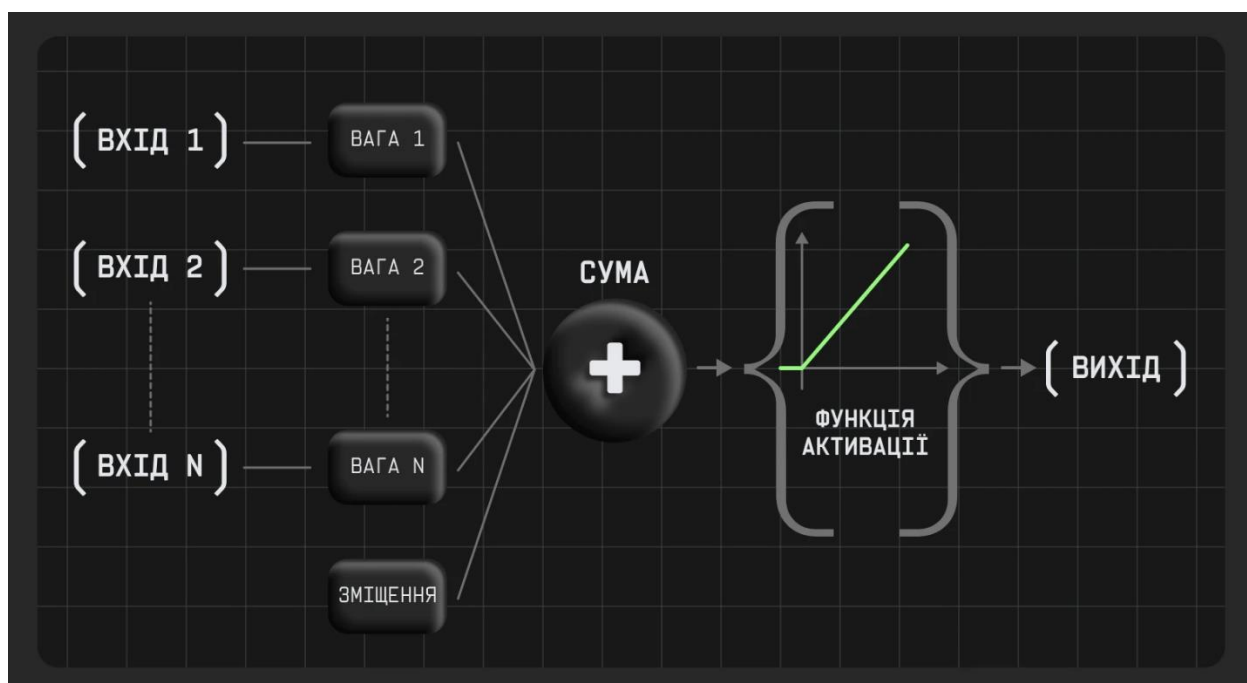
1. **Прямий прохід**: дані проходять через усі шари $\rightarrow$ отримуємо прогноз.
2. **Обчислення похибки**: різниця між прогнозом і правильним значенням.
3. **Зворотний прохід**: похибка поширюється назад через шари, коригуючи ваги за допомогою градієнтного спуску.

MLP складається щонайменше з трьох шарів взаємопов'язаних вузлів, які називаються нейронами:

- **Вхідний шар**: Отримує початкові дані, наприклад, пікселі зображення або слова речення.
- **Прихований шар (шари)**: це робочі конячки мережі, де відбуваються складні обчислення та вилучення ознак. Прихованих шарів може бути один або декілька, кожен з яких має різну кількість нейронів.
- **Вихідний шар**: Видає остаточний прогноз або результат, заснований на обробці, виконаній на попередніх шарах.



Можна ситуацію з MLP представити таким чином:



Навіщо потрібен прихований шар (Hidden layer) в MLP?

1. Подолання обмежень одношарового перцептрона

- Одношаровий перцептрон може розділити тільки **лінійно роздільні задачі** (наприклад, AND, OR).
- Але задачі на кшталт **XOR**, розпізнавання образів, мовлення, прогнозування часу — **нелінійні**.
- Прихований шар вводить **нелінійність** і дозволяє будувати складні залежності.

2. Перетворення ознак

Приховані нейрони створюють **нові представлення даних**.

- Вхід: «сирі» дані (пікселі зображення, значення сенсорів тощо).
- Прихований шар: комбінує їх у більш осмислені патерни (наприклад, контури, форми).
- Вихідний шар: працює вже з цими узагальненими ознаками.

3. Приклади

- **Зображення**: приховані шари першими вчаться знаходити прості лінії, потім — контури, і лише потім — складні об'єкти.

4. Формулювання

Можна сказати так:

- **Без прихованого шару** — мережа $\approx$ проста лінійна модель.
- **З прихованими шарами** — мережа $\approx$ універсальний апроксиматор, здатний відобразити будь-яку функцію.

[CMM LEC 2-2]

програма представляє 10 випадкових зображень недостатнього ступеня розпізнавання

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. Завантаження датасету CIFAR-10
# -----

(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.cifar10.load_data()

# Нормалізація
x_train = x_train / 255.0
x_test = x_test / 255.0

# Перетворюємо мітки в одновимірний вектор
y_train = y_train.flatten()
y_test = y_test.flatten()

# -----
# 2. Побудова моделі MLP
# -----

model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(32, 32,
3)),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(10,
activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```

# -----
---
# 3. Навчання моделі
# -----
---
model.fit(x_train, y_train, epochs=5,
batch_size=64, validation_split=0.1)

# -----
---
# 4. Класи CIFAR-10
# -----
---
class_names = [
    "airplane", "automobile", "bird", "cat",
    "deer",
    "dog", "frog", "horse", "ship", "truck"
]

# -----
---
# 5. Відображення 10 випадкових зображень +
прогноз
# -----
---
num_images = 10
indices = np.random.choice(len(x_test),
num_images, replace=False)

plt.figure(figsize=(15, 6))

for i, idx in enumerate(indices):
    img = x_test[idx]
    true_label = class_names[y_test[idx]]

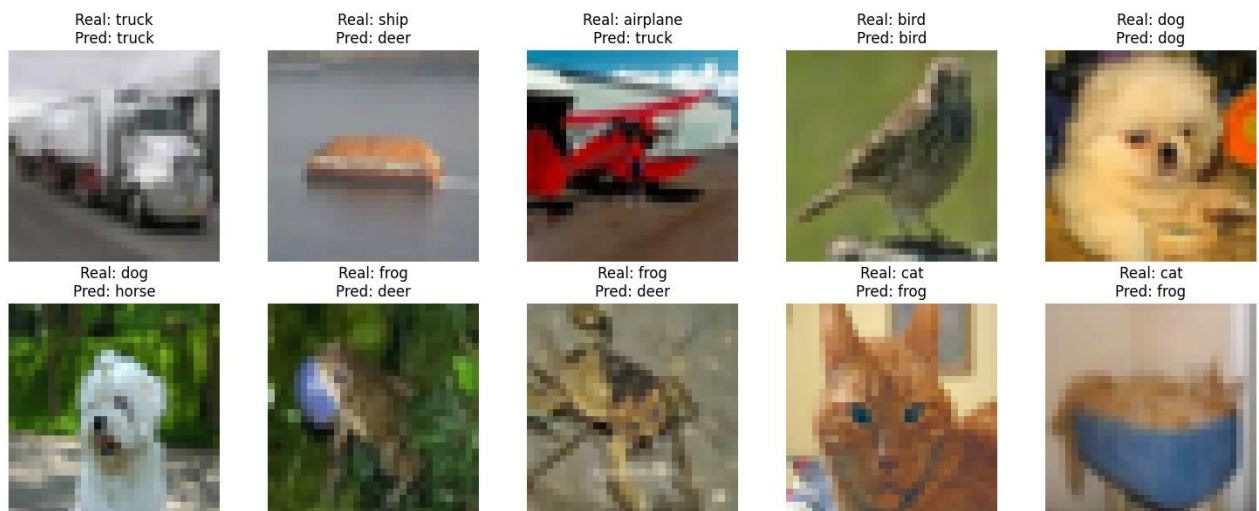
    pred = model.predict(img.reshape(1, 32, 32,
3), verbose=0)
    pred_label = class_names[np.argmax(pred)]

    plt.subplot(2, 5, i + 1)
    plt.imshow(img)

```

```
plt.axis("off")
plt.title(f"Real: {true_label}\nPred:
{pred_label}")

plt.tight_layout()
plt.show()
```



4. КОНВОЛЮЦІЙНІ НЕЙРОННІ МЕРЕЖІ (CNN – CONVOLUTIONAL NEURAL NETWORKS)

Ці мережі використовують згорткові (convolutional) шари для виділення просторових ознак зображень. Застосовуються для розпізнавання зображень, обробка відео, медичні зображення.

<https://cutt.ly/YrpT37fV>

Конволюційні нейронні мережі (**Convolutional Neural Networks, CNN**) стали основою сучасного **комп'ютерного зору**: класифікації зображень, виявлення об'єктів, сегментації, розпізнавання облич тощо. Ці мережі є різновидом нейронних мереж, що спеціалізуються саме на обробці зображень. CNN використовуються в завданнях комп'ютерного зору: генерації і класифікації

зображень, розпізнавання об'єктів і поз тощо. Ці завдання раніше намагались вирішити як за допомогою класичних нейронних мереж, тобто багатошарового перцептрона Multilayer Perceptron (MLP), так і різними евристичними методами.

Щоб краще зрозуміти суть CNN, спочатку зануримось у світ класичних алгоритмів обробки зображень. Зосередимося на задачі фільтрації зображень, яка є одним з ключових аспектів обробки зображень.

Ми всі користувались фільтрами чи то в редакторах на телефоні, чи то в додатках на кшталт Instagram. Вони використовуються для виділення або, навпаки, приховування певних характеристик зображення. Наприклад, фільтр може допомогти підкреслити контури об'єктів або зробити зображення більш розмитим. Що стосується евристичних методів, тут варто згадати алгоритми **SIFT (Scale-Invariant Feature Transform)** і **SURF (Speeded Up Robust Features)**. Вони широко використовувались для виявлення й опису локальних особливостей на зображеннях, таких як лінії, краї ліній, розпізнавання об'єктів та визначення ключових точок. Ці алгоритми допомагали в розпізнаванні об'єктів незалежно від масштабу, орієнтації та освітлення. *Однак ці методи мають свої обмеження, особливо при роботі зі складнішими зображеннями та задачами розпізнавання образів. Це призвело до пошуку нових підходів, зокрема розробки згорткових нейронних мереж.*

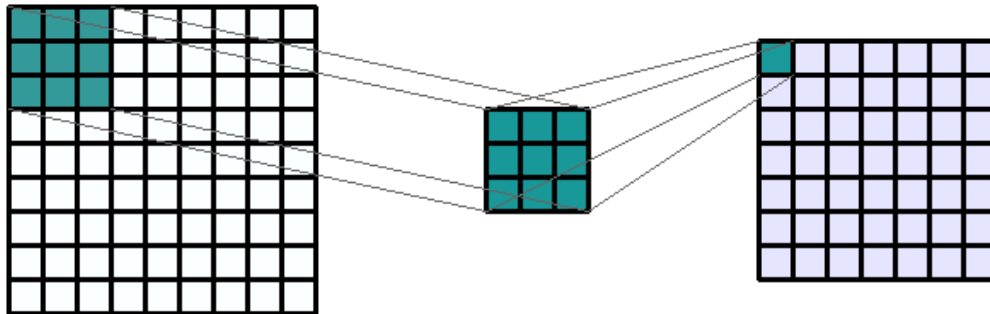
CNN у своїй основі мають подібні фільтри (і як правило не один, а одразу набір різних фільтрів). Накладаючись один за одним на зображення, вони дозволяють отримувати різні ознаки, такі як вертикальні лінії, горизонтальні лінії, різні криві тощо.

Принцип дії CNN базується на двох фундаментальних принципах. Ці принципи носять назву – *згортки і пулінгу*.

Основним елементом, що відрізняє CNN від інших типів нейронних мереж, є операція згортки — потужний інструмент, який дозволяє мережі ефективно виявляти корисні ознаки на зображеннях незалежно від їхнього положення. Згортка в CNN — це процес застосування фільтра (або «ядра згортки») до зображення. Фільтр — це невелика матриця, яку ми перемножуємо

з пікселями зображення. Він рухається по зображенню, розглядаючи невеликі ділянки зображення по черзі, що дозволяє аналізувати локальні ознаки.

Як діє згортка? Фактично вона виконує роль фільтра.



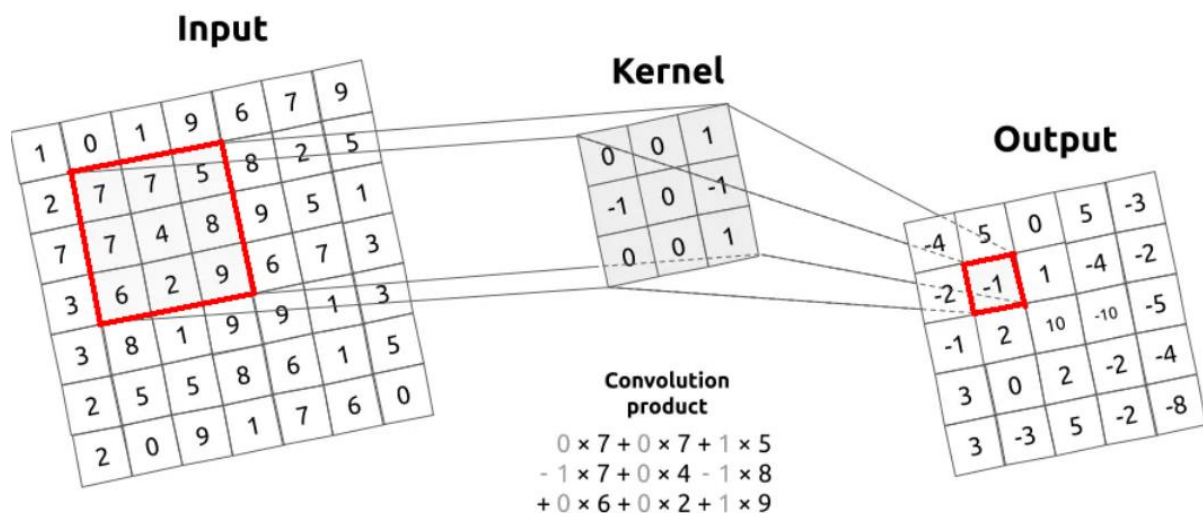
ВХІДНЕ ЗОБРАЖЕННЯ

ФІЛЬТР

РЕЗУЛЬТУЮЧЕ ЗОБРАЖЕННЯ

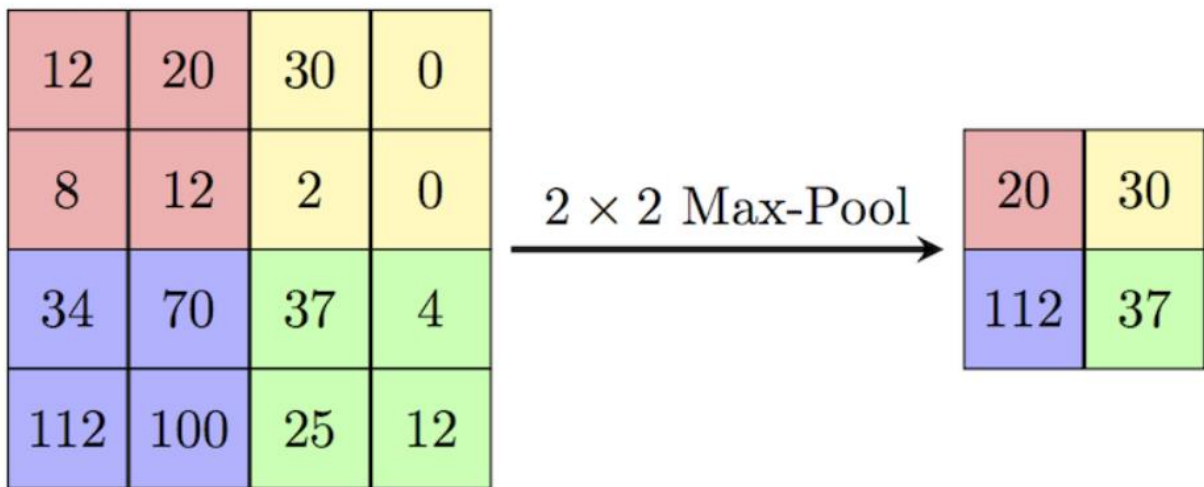
(KERNEL)

Зондування зображення



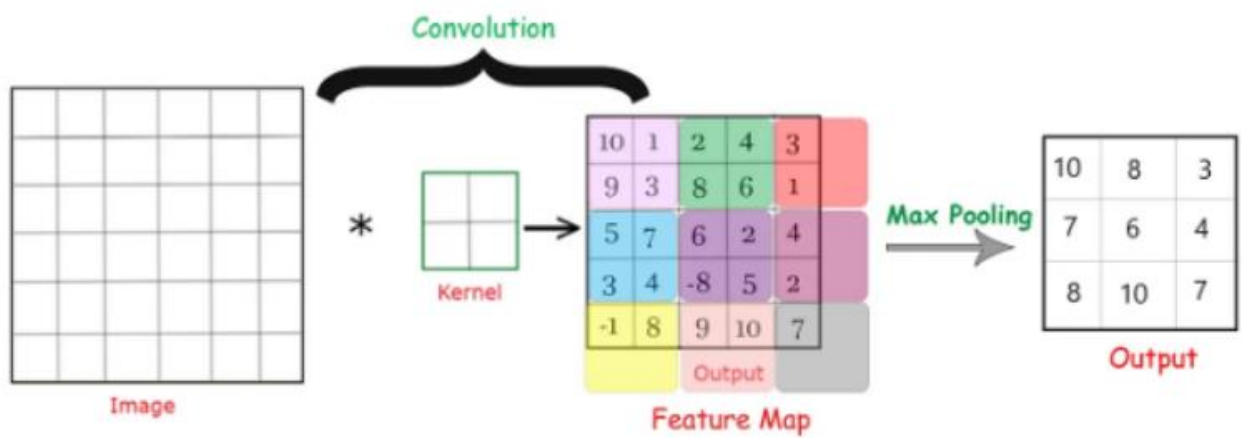
Input – вхідне зображення; **Kernel** – фільтр; **Output** – вихідне зображення.

Операція *пулінгу* в згорткових нейронних мережах є другою частиною обробки зображень. Суть цієї обробки – максимізація певної ділянки зображення. Попередній шар отримував ознаки через фільтрацію певної ділянки вхідного зображення. Наступний шар обробляє ділянки зображення отриманого від фільтрації. Робиться це наступним чином: обирається певна ділянка попереднього результату й знаходиться її максимум, що буде виходом з поточного шару. Так робимо для кожної ділянки. Сама операція виглядає так:

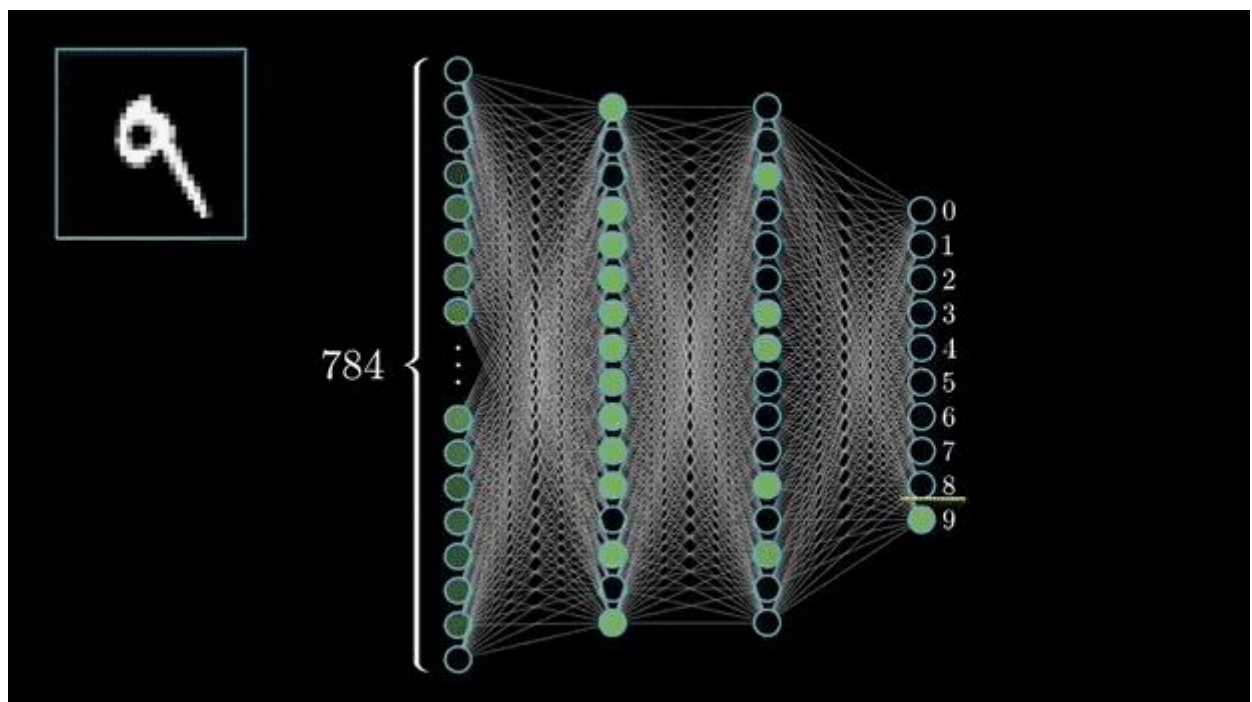


Пулінг виконується фільтром 2×2 . Для кожної області, яку обробляє фільтр, обирається найбільше значення як результат роботи пулінг-фільтру.

Фактично, це є опис одного шару згорткової нейронної мережі: перший крок — це згортка зображення накладанням на нього фільтру, другий крок — максимізація ділянок на отриманому виході.



Послідовні операції обробки зображення – згортка і пулінг.



Застосування CNN

- Класифікація зображень (ImageNet, CIFAR-10, MNIST).

- Розпізнавання облич (FaceID, FaceNet).
- Медична діагностика (аналіз рентгенів, МРТ).
- Автономні автомобілі (виявлення пішоходів, дорожніх знаків).
- Генерація зображень (GAN).

Переваги та обмеження CNN

Переваги:

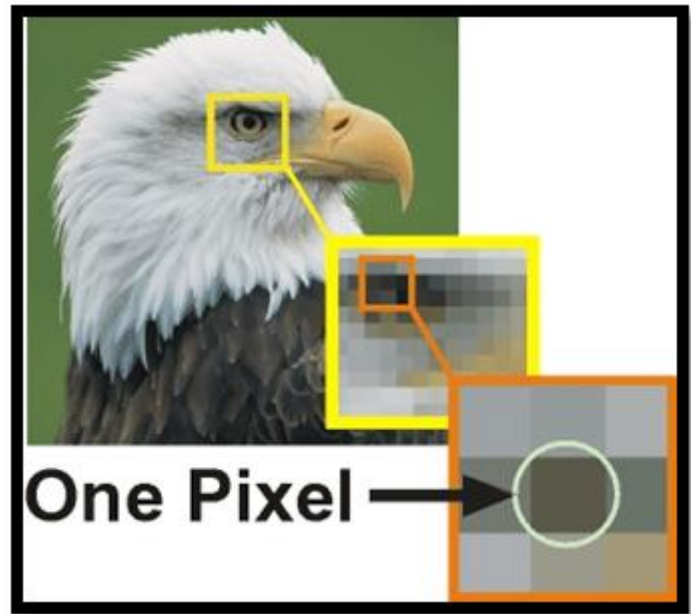
- Автоматичне виділення ознак.
- Висока точність у завданнях комп'ютерного зору.
- Можливість працювати з великими наборами даних.

Недоліки:

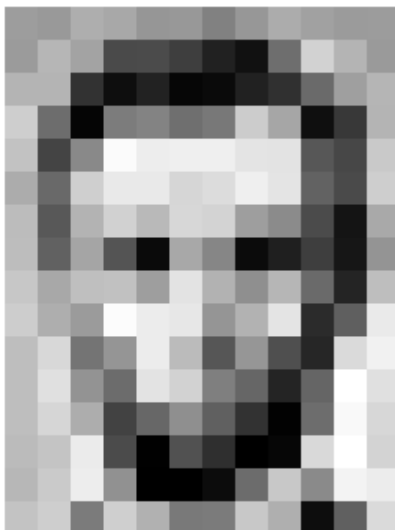
- Потребують великих обчислювальних ресурсів (GPU).
- Чутливі до якості та різноманітності навчальних даних.
- "Чорна скринька" — важко інтерпретувати, які саме ознаки мережа вивчила.

ІДЕЯ РОБОТИ CNN НА ПРОСТОМУ ПРИКЛАДІ

Цифрові зображення – це, по суті, сітки крихітних одиниць, які називаються пікселями. Кожен піксель являє собою найменшу одиницю зображення та містить інформацію про колір та інтенсивність у цій конкретній точці.



Зазвичай кожен піксель складається з *трьох значень*, що відповідають червоному, зеленому та синьому (RGB) кольоровим каналам. Ці значення визначають колір та інтенсивність цього пікселя. *Отже, звичайне зображення — це матриця пікселів*. Завдання CNN – навчитися знаходити у цій матриці ознаки (фічі): спочатку прості (лінії, кути), потім складніші (очі, обличчя, автомобіль). На відміну від цього, на зображенні у градаціях сірого кожен піксель має *одне значення*, яке представляє інтенсивність світла в цій точці. Зазвичай коливається від чорного (0) до білого (255).



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	85	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	85	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Зображення у градаціях сірого

Принцип роботи крок за кроком

1. Вхідне зображення

Припустимо, ми маємо зображення.

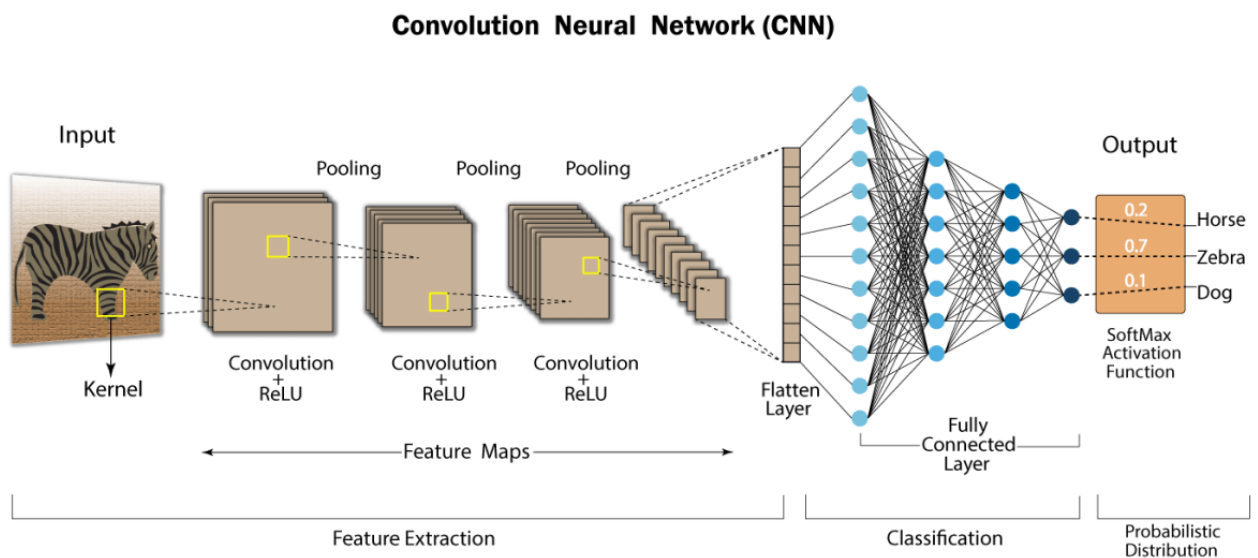
2. Згортка (Convolution)

- На зображення накладається маленький фільтр (ядро, наприклад 3×3).
- Фільтр "ковзає" по зображенню і множить значення пікселів на свої коефіцієнти.
- Результат — **карта ознак**, яка показує, де на зображенні є певний патерн (наприклад, вертикальна лінія).

Таким чином CNN автоматично "вчиться" шукати контури, лінії чи вигини.

"Мережа крок за кроком вчиться виділяти зображення: від ліній → до форм → до об'єктів".

Підсумковий результат роботи CNN



[AI LEC 5-2]

#Програма створює чорно-біле зображення із вхідного jpg-файлу, який завантажений у код програми. Програма також виводить образи *convolution*, *activation* and *pooling*

```

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from itertools import product

# set the param
plt.rc('figure', autolayout=True)
plt.rc('image', cmap='magma')

# define the kernel
kernel = tf.constant([[[-1, -1, -1],
                        [-1, 8, -1],
                        [-1, -1, -1],
                        ]])

# load the image
image = tf.io.read_file('Ganesh.jpg')
image = tf.io.decode_jpeg(image, channels=1)
image = tf.image.resize(image, size=[300, 300])

# plot the image
img = tf.squeeze(image).numpy()
plt.figure(figsize=(5, 5))
plt.imshow(img, cmap='gray')
plt.axis('off')
plt.title('Original Gray Scale image')
plt.show();

# Reformat
image = tf.image.convert_image_dtype(image, dtype=tf.float32)
image = tf.expand_dims(image, axis=0)
kernel = tf.reshape(kernel, [*kernel.shape, 1, 1])
kernel = tf.cast(kernel, dtype=tf.float32)

# convolution layer
conv_fn = tf.nn.conv2d

image_filter = conv_fn(
    input=image,
    filters=kernel,
    strides=1, # or (1, 1)
    padding='SAME',
)

plt.figure(figsize=(15, 5))

# Plot the convolved image
plt.subplot(1, 3, 1)

plt.imshow(
    tf.squeeze(image_filter)
)
plt.axis('off')
plt.title('Convolution')

# activation layer
relu_fn = tf.nn.relu
# Image detection
image_detect = relu_fn(image_filter)

plt.subplot(1, 3, 2)
plt.imshow(
    # Reformat for plotting
    tf.squeeze(image_detect)
)

```

```

plt.axis('off')
plt.title('Activation')

# Pooling layer
pool = tf.nn.pool
image_condense = pool(input=image_detect,
                      window_shape=(2, 2),
                      pooling_type='MAX',
                      strides=(2, 2),
                      padding='SAME',
                      )

plt.subplot(1, 3, 3)
plt.imshow(tf.squeeze(image_condense))
plt.axis('off')
plt.title('Pooling')
plt.show()

```

Original Gray Scale image





<https://doi.org/10.3390/electronics13245049>